CS2201 Computer Organization & Architecture

L10: Data Hazards and Mitigation Techniques



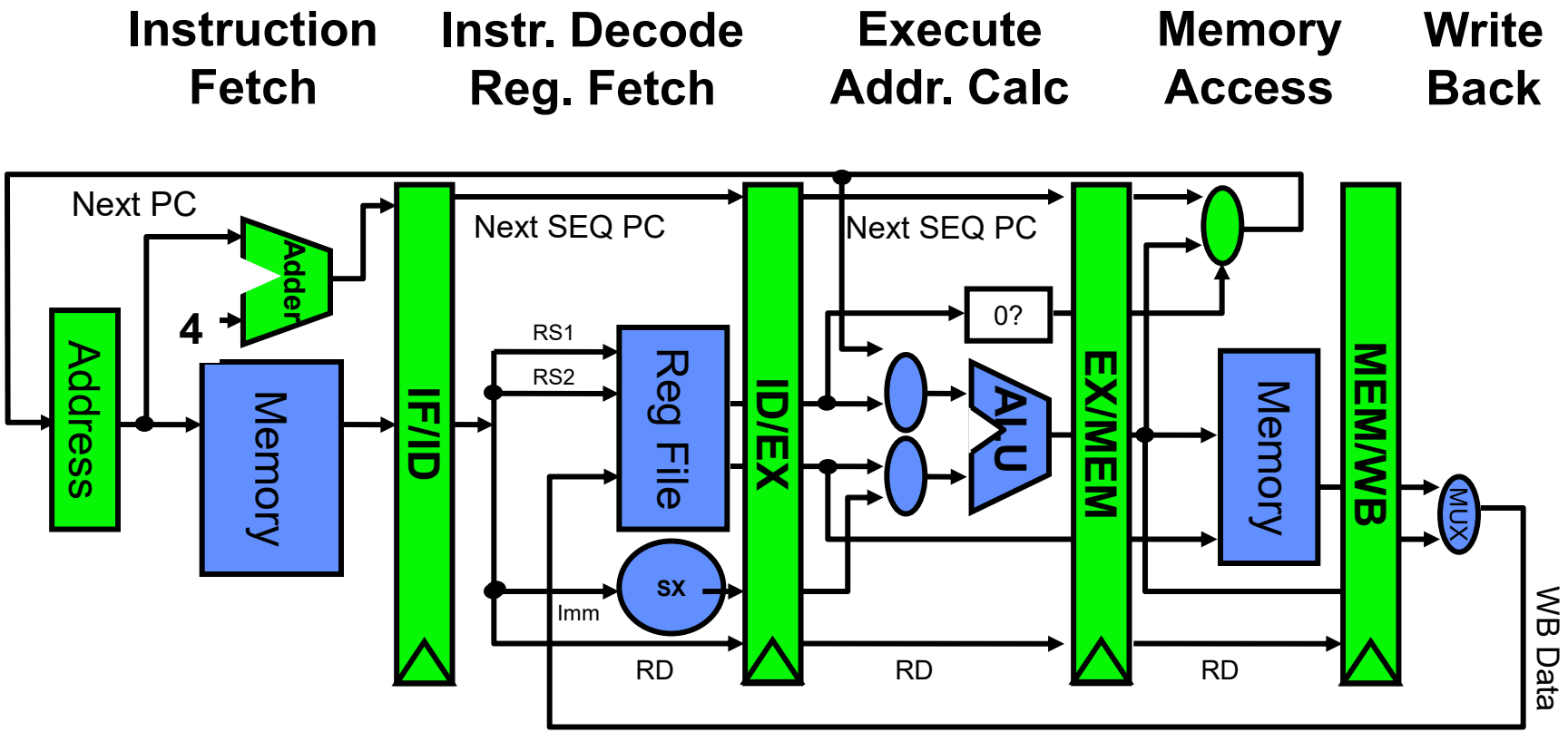
John Jose

Associate Professor

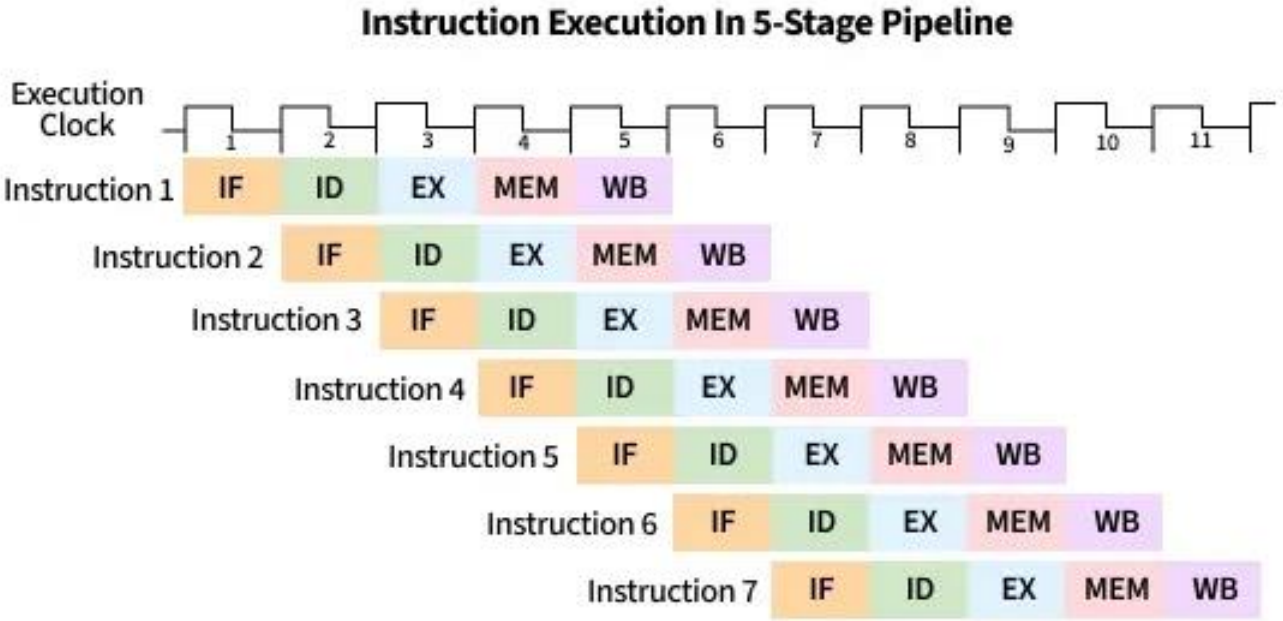
Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

5 Stage RISC Pipeline



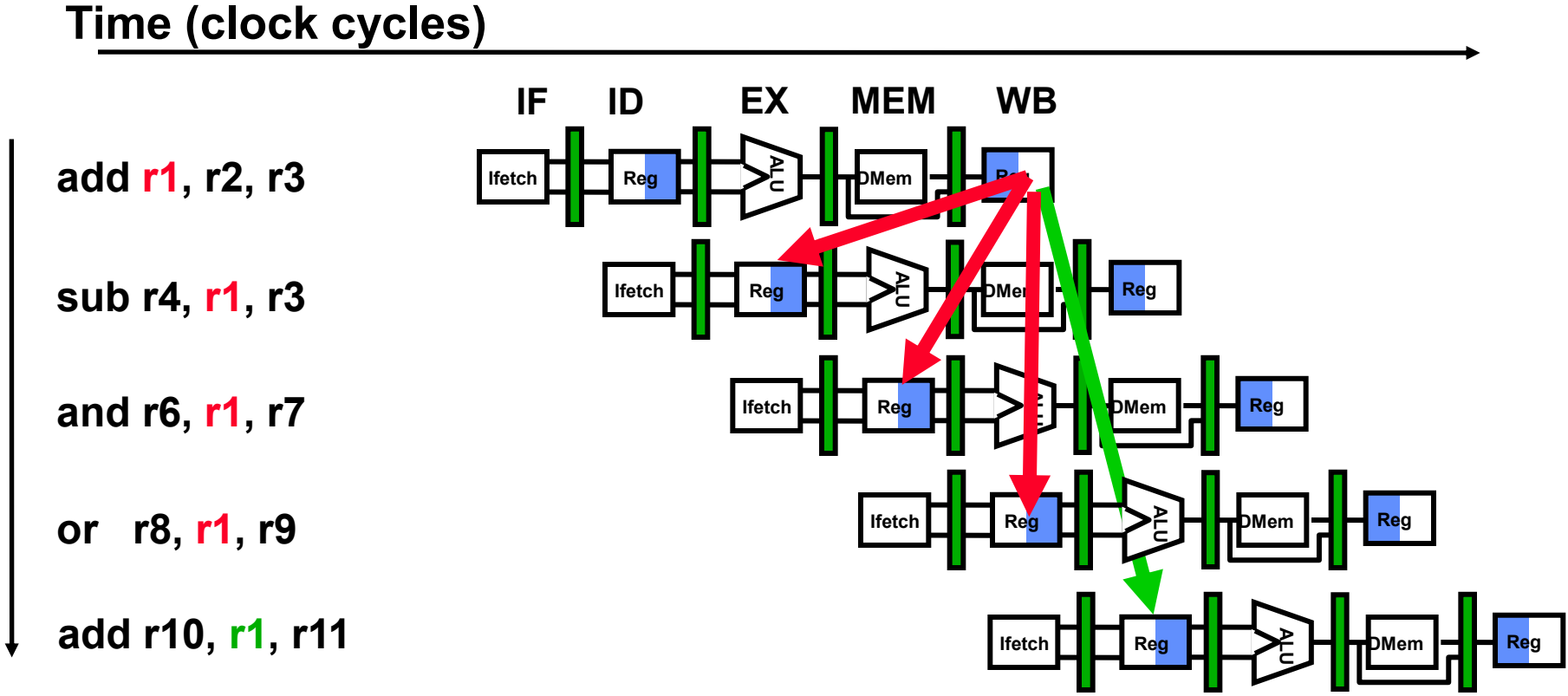
5 Stage RISC Pipeline



Limits to pipelining

- **Hazards:** circumstances that would cause incorrect execution if next instruction is fetched and executed
  - **Structural hazards:** Different instructions, at different stages, in the pipeline want to use the same hardware resource
  - **Data hazards:** An instruction in the pipeline requires data to be computed by a previous instruction still in the pipeline
  - **Control hazards:** Succeeding instruction, to put into pipeline, depends on the outcome of a previous branch instruction, already in pipeline

Data Hazard


Time (clock cycles)
IF
ID
EX
MEM
WB
add r1, r2, r3
sub r4, r1, r3
and r6, r1, r7
or r8, r1, r9
add r10, r1, r11
Ifetch
Reg
ALU
DMem
Reg

Three Generic Data Hazards

- **Read After Write (RAW)**
  $Instr_J$ tries to read operand before $Instr_I$ writes it

I : ADD R1, R2, R3

J: SUB R4, R1, R5

Instruction	CC 1	CC 2	CC 3	CC 4	CC 5	CC 6
ADD R1, R2, R3	IF	ID	EX	MEM	WB	
SUB R4, R1, R5		IF	ID	EX	MEM	WB



- Caused by a true data dependence
- This hazard results from an actual need for communication
- Pipeline Stalling (Bubbles): Delays the SUB instruction by 2 cycles until the ADD reaches the WB phase.

Three Generic Data Hazards

- **Write After Read (WAR)**
  $Instr_J$ writes operand before $Instr_I$ reads it

I: SUB R4, R1, R3

J: ADD R1, R2, R3

K: MUL R6, R1, R7

- Called an anti-dependence by compiler writers.
- This results from reuse of the name R1
- Can't happen in MIPS 5 stage pipeline because:
  - All instructions take 5 stages, and
  - Reads are always in stage 2, and
  - Writes are always in stage 5

I: SUB R4, R1, R3

J: ADD R8, R2, R3

K: MUL R6, R8, R7

Three Generic Data Hazards

- **Write After Write (WAW)**
  $Instr_J$ writes operand before $Instr_I$ writes it.

I: SUB R1, R4, R3

J: ADD R1, R2, R5

K: MUL R6, R1, R7

- Called an output dependence
- This also results from the reuse of name R1.
- Can't happen in MIPS 5 stage pipeline because:
  - All instructions take 5 stages, and
  - Writes are always in stage 5
- WAR and WAW happens in out of order pipes

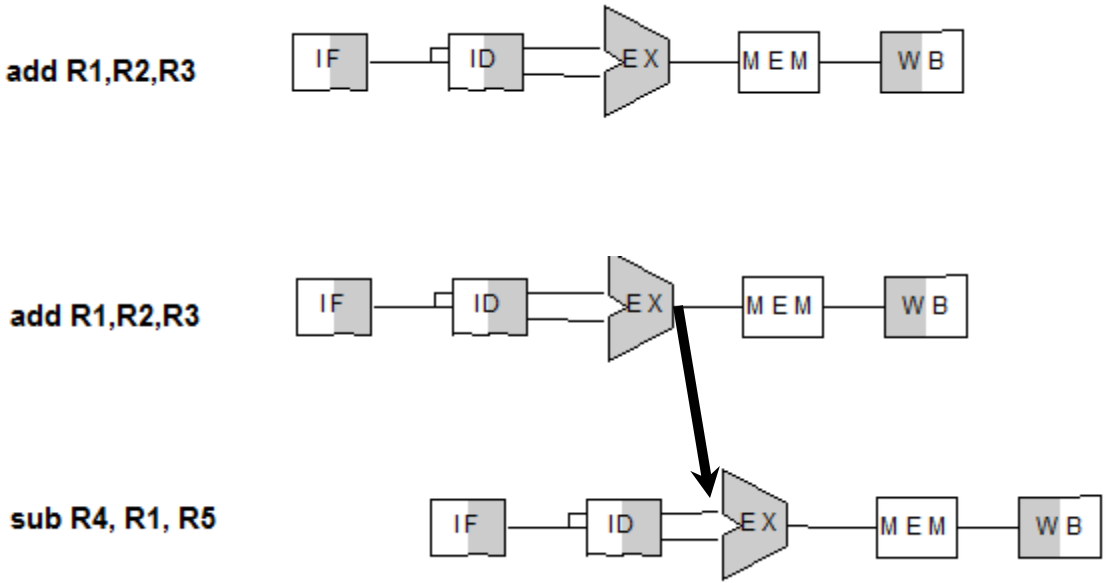
I: SUB R1, R4, R3

J: ADD R8, R2, R5

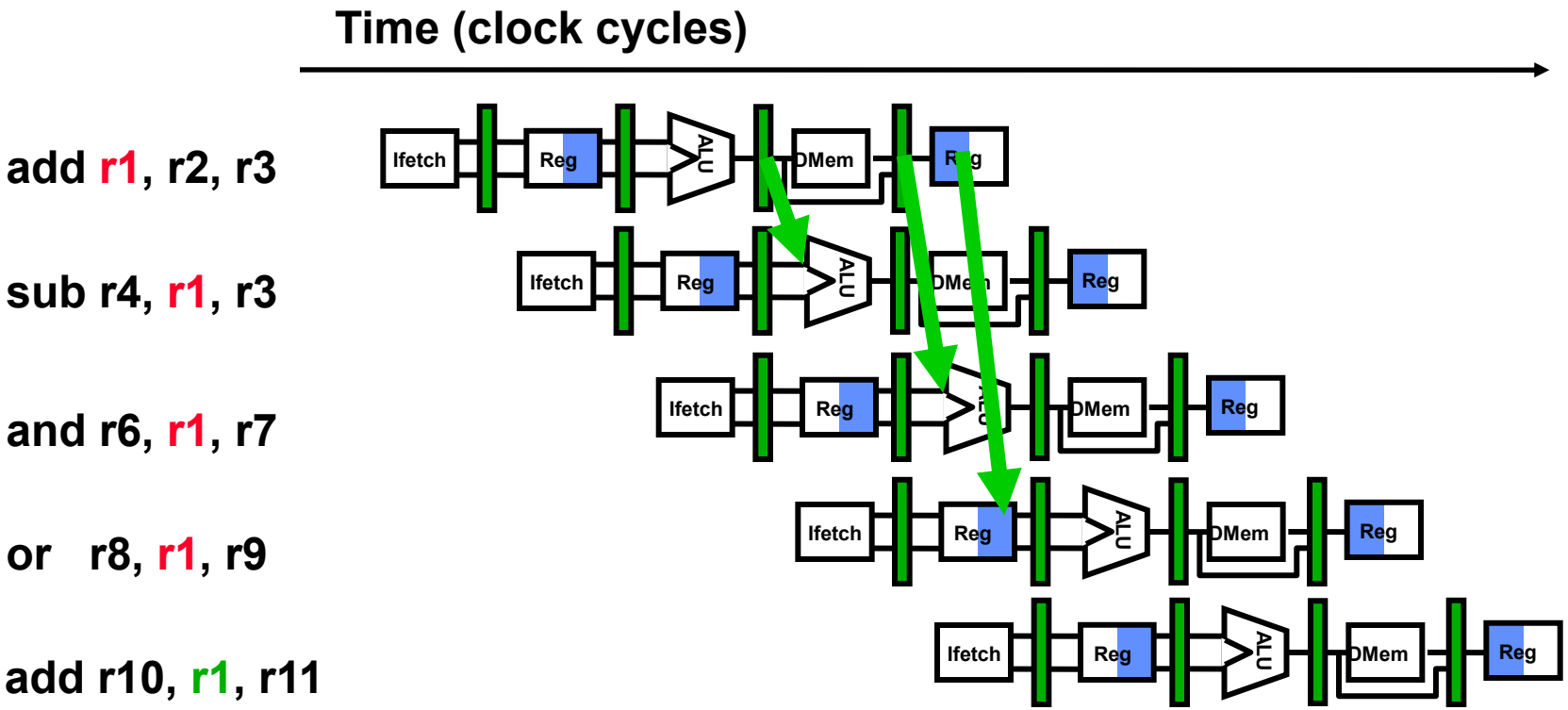
K: MUL R6, R8, R7

How to Handle Data Hazard?

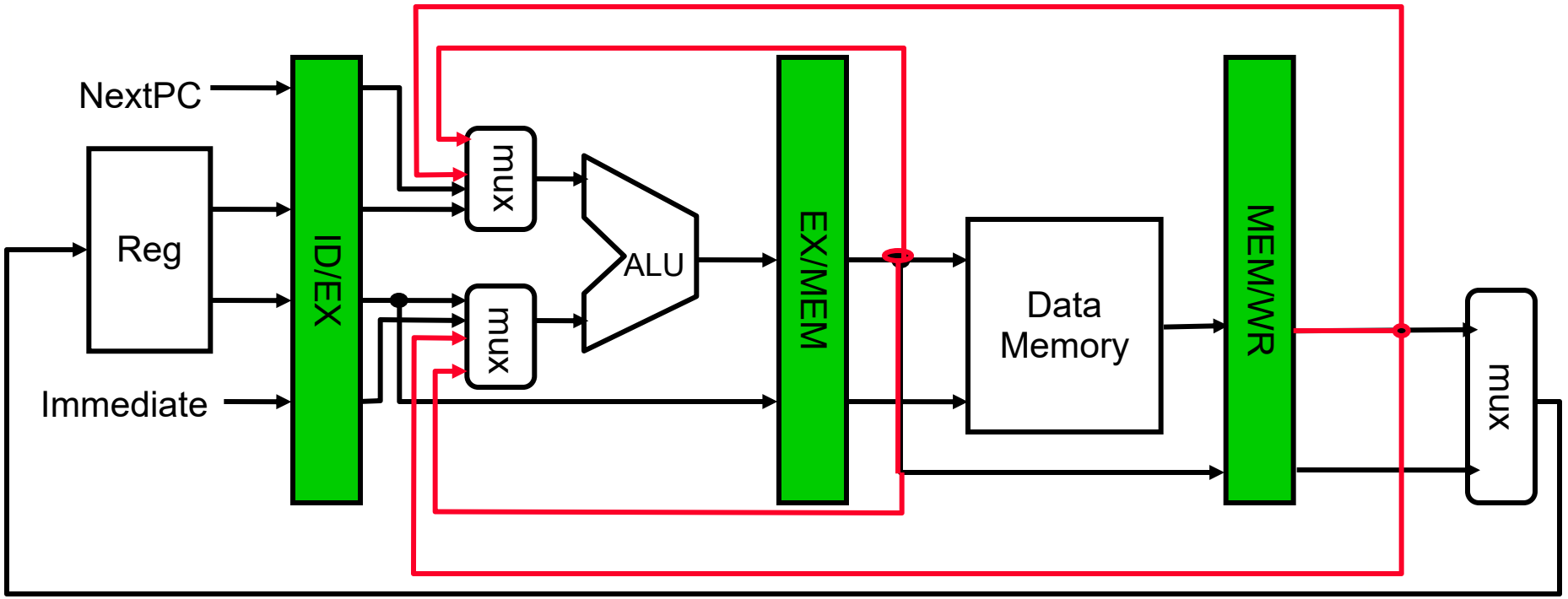
- Data hazard: instruction needs data from the result of a previous instruction still executing in the pipeline.
- **Solution:** Forward data if possible.



Operand Forwarding to Avoid Data Hazard



Hardware Change for Forwarding



Data Hazard even with Operand Forwarding

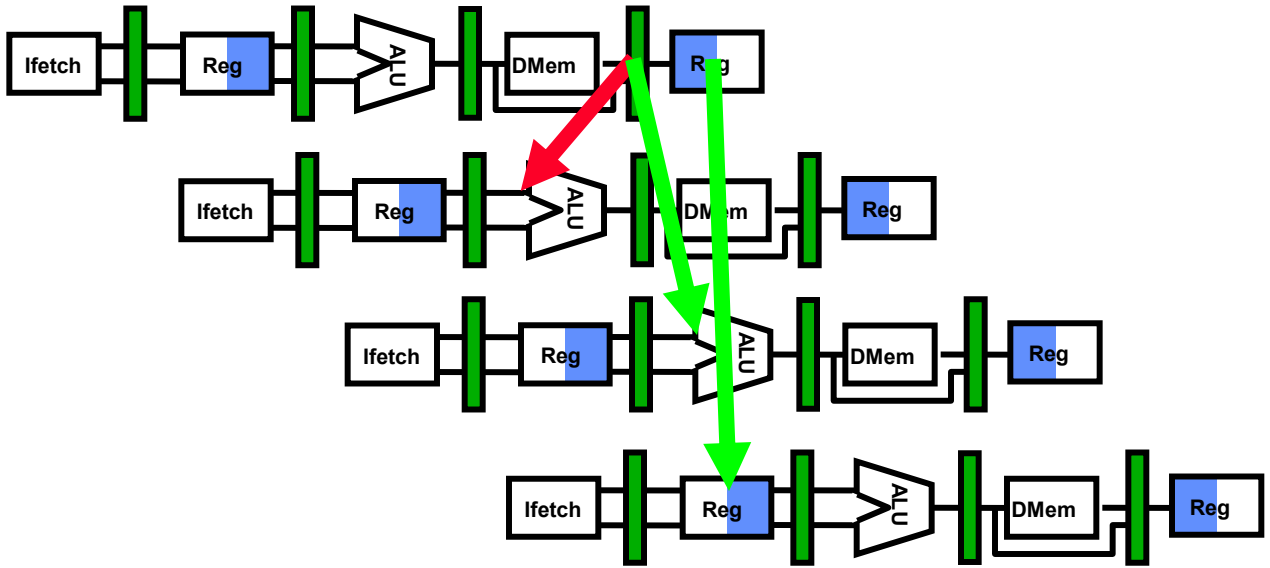
Time (clock cycles)

lw r1, 24(r2)

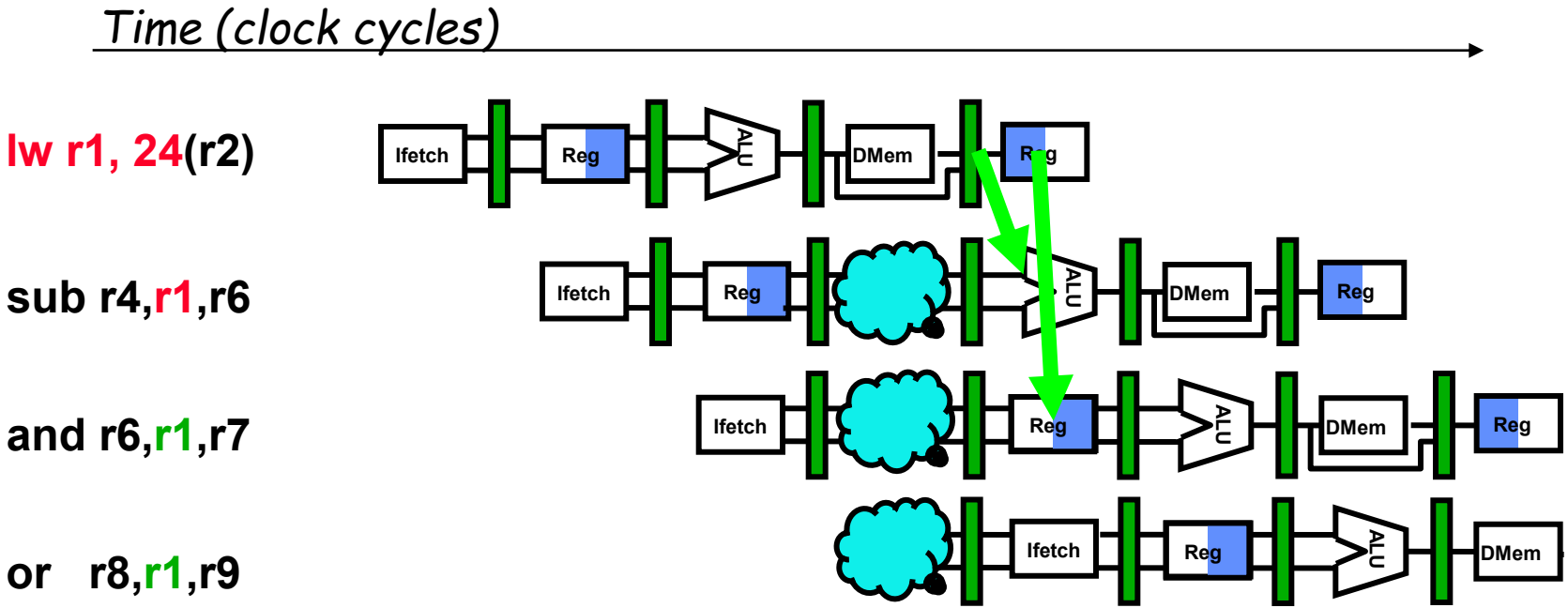
sub r4, r1, r6

and r6, r1, r7

or r8, r1, r9



Resolving the Load-ALU Hazard

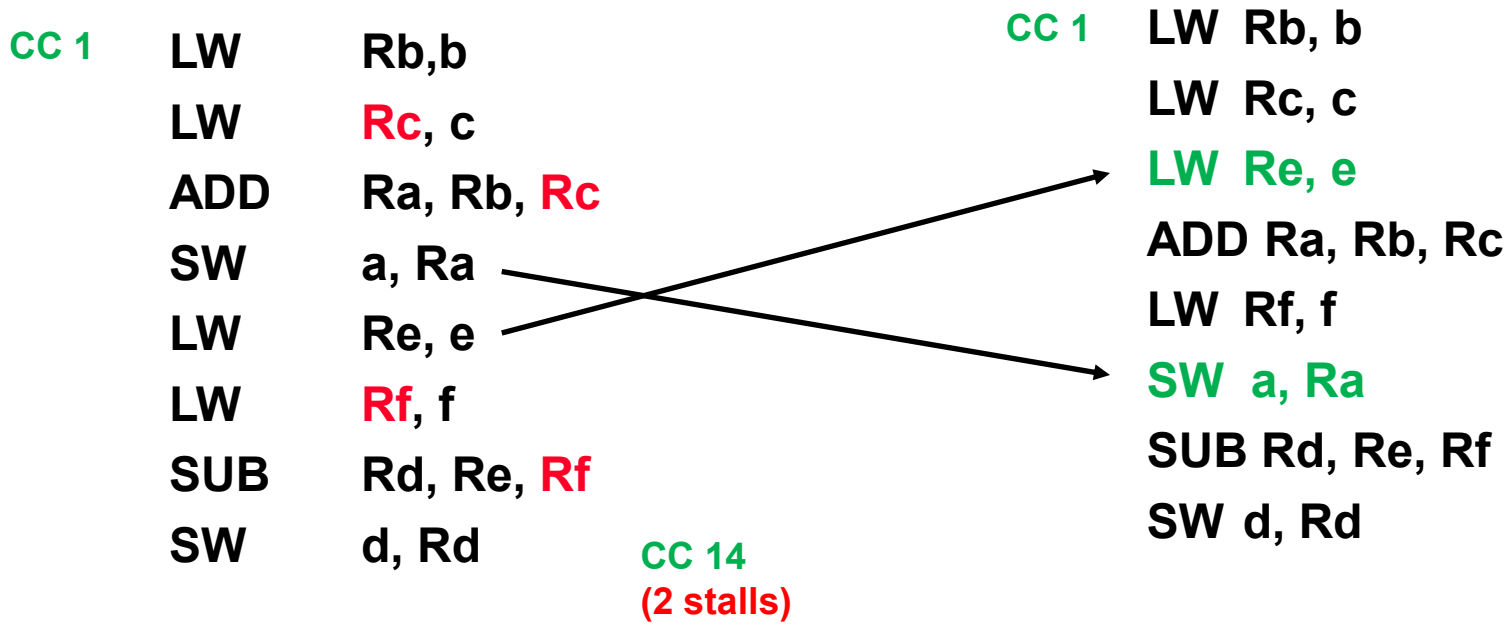


Software Scheduling for Data Hazards

Assume a, b, c, d ,e, and f are in memory.

a = b + c;

d = e – f;



Pipeline Stalls Example

Given a non-pipelined architecture running at 2 GHz, that takes 5 cycles to finish an instruction. You want to make it pipelined with 5 stages. Due to hardware overhead the pipelined design will operate only at 1 GHz. On an average, 5% of instructions cause a stall of 20 cycles due to pipeline hazards. What is the speedup obtained by the pipelined design over the non-pipelined design?

- Speedup from pipelining = ETI_unpipelined / ETI_pipelined
- CPI_unpipelined = 5, CC_unpipelined → 2GHz = 0.5ns
- CC_pipelined = 1GHz = 1ns
- CPI_pipelined = Ideal CPI + Pipeline stalls cycles/ instruction

  = 1 + 0.05 * 20 = 2

Pipeline Stalls Example

- Speedup from pipelining = ETI_unpipelined / ETI_pipelined
- CPI_unpipelined = 5, CC_unpipelined → 2GHz = 0.5ns
- CC_pipelined = 1GHz = 1ns
- CPI_pipelined = Ideal CPI + Pipeline stalls cycles/ instruction

  = 1 + 0.05 * 20 = 2
- ETI_unpipelined = CPI_unpipelinedxCC_unpipelined = 5x0.5 =2.5 ns
- ETI_pipelined = CPI_pipelinedxCC_pipelined = 2x1 =2 ns
- Speedup from pipelining = ETI_unpipelined / ETI_pipelined

  = 2.5 ns / 2 ns = 1.25 times



johnjose@iitg.ac.in

http://www.iitg.ac.in/johnjose/

https://www.linkedin.com/in/john-jose-bb8431106/